

COMPUTATIONAL CONTROL

Lecture Handout

April 9, 2026

Contents

1	Dynamic Programming and LQR	2
1.1	Introduction	2
1.2	Dynamic Programming	3
1.3	Optimal LQR	4
1.4	Dynamic Programming for LQR	4
1.4.1	Infinite-Horizon LQR	6
2	Model Predictive Control	10
2.1	Introduction	10
2.2	Implementation of MPC	14
2.2.1	Explicit MPC	14
2.3	Stability	17
2.4	Disturbance rejection	22
2.4.1	Zero steady-state offset and incremental MPC	23
2.4.2	Disturbance rejection during transients	24
2.4.3	Advanced disturbance models	25
3	Economic MPC	27

1 Dynamic Programming and LQR

1.1 Introduction

In general we formulate the optimization control task as

$$\min_{\mathbf{u} \in \mathcal{U}} J(\mathbf{u}) \quad (1)$$

where $\mathcal{U} = \tilde{\mathcal{U}} \times \{0, 1, \dots, T\}$ is the set of the feasible inputs and $J(\mathbf{u})$ is generally the sum of some *stage costs* and a terminal cost/constraint.

The issue is that generally 1 is an intractable optimization problem. This is due to several factors:

- the dimension of $\mathbf{u}$ is too big;
- the cost function requires an accurate model of the system;
- the optimization problem is **non-convex**;
- In presence of **disturbances** and **uncertainty** the open-loop nature of the task makes it useless

In this context, Dynamic Programming (DP) is a powerful tool whenever we can decompose the problem. In particular when the problem allows for a **Markovian representation**:

Key assumptions: Markovian representation

The following hypothesis are crucial in order for DP to be meaningful:

1. There exists a **Markovian State**^a that evolves according to some dynamics

$$x_{t+1} = f_t(x_t, u_t)^b$$

2. The initial condition x_0 is known.

3. The cost is additive over time, i.e.

$$J = \sum_t g_t(x_t, u_t)$$

^ai.e. $X_{t+1} \perp X_{1:t-1} | X_t$

^bdiscrete time dynamical system representation. Notice f_t can change with time

In this context a powerful result is given by **Bellman's principle**. Considering the following setup:

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{x}} \quad & \sum_{t=0}^T g_t(x_t, u_t) \\ \text{subject to} \quad & x_{t+1} = f_t(x_t, u_t) \\ & x_0 = X_0 \\ & x_t \in \mathcal{X}_t \quad \forall t \\ & u_t \in \mathcal{U}_t \quad \forall t. \end{aligned}$$

a very powerful result on which relies DP is **Bellman's optimality principle**.

Bellman's optimality principle

An optimal policy has the property that whatever the initial state and the initial decisions are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.

In an intuitive way we can say that any tail of an optimal trajectory is optimal too.

$$\begin{aligned} V_0(X_0) &= \min_{u_0} \left\{ g_0(X_0, u_0) + \min_{\substack{x_1, \dots, x_T \\ u_1, \dots, u_T}} \sum_{t=1}^T g_t(x_t, u_t) \text{ s.t. } \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_1 = f_0(X_0, u_0) \end{cases} \right\} \\ &= \min_{u_0} \{g_0(X_0, u_0) + V_1(x_1)\} \\ &= \min_{u_0} \{g_0(X_0, u_0) + V_1(f_0(X_0, u_0))\} \end{aligned}$$

We can see that the problems are nested, therefore we need to proceed in a **backward** way, starting from the last time step and going backward in time. In particular we need to proceed in the following order

1. solve the subproblem parametrized by x_1 ;
2. compute the value function V_1 ;
3. solve the outer problem

$$\min_{u_0} \{g(X_0, u_0) + V_1(f_0(X_0, u_0))\}$$

1.2 Dynamic Programming

We can now write the general DP algorithm, which is a backward procedure to compute the value function and the optimal policy. We start from the last time step and we proceed backward in time until we reach the initial time step.

At the last stage (T) the subproblem is trivial, since there are no more decisions to be taken, therefore we have

$$V_T(x_T) = g_T(x_T)$$

At time $T - 1$ we compute the value function as

$$\min_{u_{T-1}} \{g_{T-1}(x_{T-1}, u_{T-1}) + V_T(f_{T-1}(x_{T-1}, u_{T-1}))\}$$

In particular notice that in the formulation above we know the value of $V_T(x_T)$, therefore we can compute the value function at time $T - 1$ and so on until we reach the initial time step. So at this moment we have an optimal policy for the time step $T - 1$, that means that we know how to compute the optimal control action at time $T - 1$ given the state at time $T - 1$. We can repeat this procedure until we reach the initial time step, therefore we have an optimal policy for all time steps.

At time $T - 2$ we compute $V_{T-2}(x_{T-2})$ as

$$\min_{u_{T-2}} \{g_{T-2}(x_{T-2}, u_{T-2}) + V_{T-1}(f_{T-2}(x_{T-2}, u_{T-2}))\}$$

Notice that this value function $V_{T-2}(x_{T-2})$ is a function of the state x_{T-2} , therefore we have to compute it $\forall x_{T-2} \in \mathcal{X}_{T-2}$, which is generally a very hard task.

Problem Decomposition

We have seen that the optimal control problem is decomposed into **stage problems** that we can solve via **backward induction** :

$$V_t(x_t) = \min_{u_t} \{g_t(x_t, u_t) + V_{t+1}(f_t(x_t, u_t))\} \quad (2)$$

And $V_T(x_T) = g_T(x_T)$

A reasonable question at this point would be, in which cases is this stage problem formulation simple to solve ? We can list three scenarios:

- Decision space $\tilde{\mathcal{U}}$ is convex , or maybe finite.
- If g_t is convex in u_t .
- If V_{t+1} evaluated at the future step f_t is convex in u_t .

In practice, a very common setup in which the assumptions above are satisfied is when we have a **quadratic cost** g_t and a **linear dynamics** f_t , which is the so called **LQR problem**. Another way is also to use approximations of the value function, in which case maybe not all the assumptions above are satisfied, but we can still solve the stage problem.

1.3 Optimal LQR

The LQR problem is a special case of the optimal control problem, in which we have a linear dynamics and a quadratic cost. In particular we have:

- Markovian update and linear dynamics

$$x_{t+1} = A_t x_t + B_t u_t$$

- Quadratic cost function:

$$\sum_{t=0}^{T-1} \left(x_t^\top Q_t x_t + u_t^\top R_t u_t \right) + x_T^\top S x_T \quad \text{with } Q, S \geq 0, R > 0$$

- Value function (i.e. minimum cost to go starting from x at time t) :

$$V_t(x) = \min_{u_t, \dots, u_{T-1}} \sum_{k=t}^{T-1} \left(x_k^\top Q_k x_k + u_k^\top R_k u_k \right) + x_T^\top S x_T$$

$$\begin{aligned} \text{subject to } x_{k+1} &= A_k x_k + B_k u_k \quad \forall k = t, \dots, T-1 \\ x_t &= x \end{aligned}$$

We say that the optimal cost is $V_0(x_0)$.

1.4 Dynamic Programming for LQR

Let us now specialize the dynamic programming recursion to the Linear Quadratic Regulator (LQR) problem. Recall that the value function represents the optimal *cost-to-go*, i.e. the minimum cost achievable from a given state when acting optimally from that point onward.

A key observation in the LQR setting is that the terminal cost is quadratic in the state. Indeed, from the problem formulation we have

$$V_T(x) = x^\top S x,$$

which is a convex quadratic function since $S \succeq 0$.

The main idea of the LQR derivation is to show that the value function $V_t(x)$ is also quadratic and has the form of $V_t(x) = x^\top P_t x$ for some symmetric positive semidefinite matrix P_t .

We will then show that the matrices P_t can then be computed recursively by working *backward in time* starting from the terminal time T . Eventually the the optimal control input u_t will be computed as a solution of a convex quadratic optimization problem, which admits a closed-form solution.

Proof. To derive the recursion, consider the dynamic programming stage problem at time t where the value function is

$$V_t(x) = \min_u \left(x^\top Q x + u^\top R u + V_{t+1}(Ax + Bu) \right).$$

We now proceed by induction, assuming that $V_{t+1}(x) = x^\top P_{t+1} x$ for some positive semidefinite matrix P_{t+1} . Then $V_t(x)$ can be written as

$$\begin{aligned} V_t(x) &= x^\top Q x + \min_u \left(u^\top R u + (Ax + Bu)^\top P_{t+1} (Ax + Bu) \right) \\ &= x^\top Q x + \min_u \left(u^\top R u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x + u^\top B^\top P_{t+1} B u \right) \\ &= x^\top Q x + \min_u \left(u^\top (R + B^\top P_{t+1} B) u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x \right). \end{aligned}$$

We set the gradient of the expression inside the minimization with respect to u to zero:

$$(R + B^\top P_{t+1} B)u + B^\top P_{t+1} A x = 0$$

Yielding the optimal control input

$$u^* = -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x$$

But now we need to check that $V_t(x)$ is indeed quadratic in X . That means $V_t(x) = x^\top P_t x$ for some P_t and we have to find how to compute P_t . We can do that by plugging the optimal control input u^* back into the expression for $V_t(x)$:

$$\begin{aligned} V_t(x) &= x^\top Q x + (u^*)^\top R u^* + (Ax + Bu^*)^\top P_{t+1} (Ax + Bu^*) \\ &= x^\top Q x + (u^*)^\top R u^* + x^\top A^\top P_{t+1} A x + 2(u^*)^\top B^\top P_{t+1} A x + (u^*)^\top B^\top P_{t+1} B u^* \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top (R + B^\top P_{t+1} B) u^* + 2(u^*)^\top B^\top P_{t+1} A x \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top ((R + B^\top P_{t+1} B) u^* + 2B^\top P_{t+1} A x) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x \\ &\quad - ((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x)^\top (- (R + B^\top P_{t+1} B) (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x + 2B^\top P_{t+1} A x) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + - ((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x)^\top (B^\top P_{t+1} A x) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x - x^\top A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x \\ &= x^\top \left(Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \right) x \\ &= x^\top P_t x \end{aligned}$$

To complete the proof one should check that P_t is positive semidefinite. Here we omit the proof.

In conclusion we have been able to derive a closed-form expression for the optimal control input u^* for the LQR problem. Be aware that this solution is not valid anymore if the problem becomes constrained. $\square$

We summarize here the main results:

Optimal LQR solution

Backward induction

$$u_t = - \underbrace{(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A}_{\Gamma_t} x_t$$

where

$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \quad \text{and } P_T = S$$

Forward integration from x_0 :

$$x_{t+1} = Ax_t + Bu_t$$

So clearly one could compute this whole open-loop sequence $u_0, \dots, u_{T-1}$ in an offline computation. But what if the optimal control input $\mathbf{u}$ takes us to a trajectory different from the one we computed? In this case we can use the optimal policy Γ_t to compute the optimal control input at each time step, given the current state x_t . This is the main advantage of having a closed-form solution for the optimal control input, since we can use it in a feedback way.

We can do a quick comparison of the computational cost of using these equations in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**

- $n \times n$ matrix multiplications to compute P_t for $t = T-1, \dots, 0$
- $m \times m$ matrix inversions to compute Γ_t for $t = T-1, \dots, 0$
- T iterations to compute P_t and Γ_t for $t = T-1, \dots, 0$

- **Online computation**

- Storage of T $m \times n$ matrices Γ_t for $t = T-1, \dots, 0$
- $m \times m$ matrix multiplications to compute u_t at each time step, given x_t

1.4.1 Infinite-Horizon LQR

In many practical control applications the system is required to operate over a very long time horizon. Examples include regulation problems where the controller must stabilize a system indefinitely, or tracking tasks where disturbances may occur continuously.

For these reasons it is natural to study the limit case where the horizon tends to infinity. The resulting problem is known as the **infinite-horizon Linear Quadratic Regulator (LQR)**.

Consider the linear time-invariant system

$$x_{t+1} = Ax_t + Bu_t, \quad x_0 \text{ given}$$

and the infinite-horizon quadratic cost

$$\min_{u_0, u_1, \dots} \sum_{t=0}^{\infty} (x_t^\top Q x_t + u_t^\top R u_t), \quad Q \succeq 0, R \succ 0.$$

Compared to the finite-horizon case, the cost now accumulates indefinitely. Therefore a first important question concerns the **feasibility** of the problem.

Feasibility

If the pair (A, B) is **stabilizable**, then there exists a control input sequence that yields a finite value of the infinite-horizon cost.

Indeed, if the system is stabilizable there exists a feedback matrix K such that the closed-loop matrix $A + BK$ has all eigenvalues inside the unit circle. Consider the linear feedback policy

$$u_t = Kx_t.$$

The state then evolves according to

$$x_t = (A + BK)^t x_0.$$

Substituting this into the cost yields

$$V(x_0) = \sum_{t=0}^{\infty} (x_t^\top Q x_t + x_t^\top K^\top R K x_t).$$

Using the closed-loop dynamics we obtain

$$V(x_0) = x_0^\top \left(\sum_{t=0}^{\infty} (A + BK)^{t\top} (Q + K^\top R K) (A + BK)^t \right) x_0.$$

Since $A + BK$ is stable, the above summation behaves like a *matrix geometric series* and therefore converges. Consequently the cost is finite.

But how do we compute the optimal policy for the infinite-horizon LQR problem? The answer is that we can still apply the dynamic programming recursion, but now the value function does not depend on time. The solution is given by the **Discrete Algebraic Riccati Equation (DARE)**, which is a fixed-point equation for the matrix P that defines the value function.

Algebraic Riccati Equation

The iteration

$$P_{t-1} = Q + A^\top P_t A - A^\top P_t B (R + B^\top P_t B)^{-1} B^\top P_t A \quad P_0 = 0$$

converges, for $t \rightarrow -\infty$, to a unique positive semidefinite solution $P_\infty \succ 0$ of the algebraic Riccati equation

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A$$

We can show that then the optimal feedback control in the infinite-horizon case is given by

$$u_t = - \underbrace{(R + B^\top P B)^{-1} B^\top P A}_{F_\infty} x_t$$

and thus minimizes the infinite horizon cost and yields $V(X_0) = X_0^\top P X_0$.

We now show again a comparison of the computational cost of using the infinite-horizon solution in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**

- $n \times n$ matrix multiplications to compute P
- $m \times m$ matrix inversions to compute F_∞
- T iterations to compute P and F_∞ for $t = T - 1, \dots, 0$
- "Infinite iterations" to compute P for $t \rightarrow -\infty$ ¹

- **Online computation**

- Storage of a **single** $m \times n$ matrix F_∞ . So less memory is required compared to the finite-horizon case, since we do not need to store T matrices Γ_t .
- $m \times n$ matrix multiplications to compute u_t at each time step, given x_t

A natural question at this point is whether the optimal infinite-horizon feedback law is also *stabilizing*. Indeed, even if the infinite-horizon cost is well defined and the Riccati equation admits a solution, one would still like to know whether the resulting closed-loop dynamics

$$x_{t+1} = (A + BF_\infty)x_t$$

are asymptotically stable.

This is a meaningful question because the quadratic cost does *not* necessarily penalize all state directions. In particular, if some unstable state component is not “seen” by the cost, then the optimizer may have no incentive to stabilize it.

Illustrative example Consider the system

$$x_{t+1} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} x_t + u_t, \quad Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Here both open-loop modes are unstable, since the eigenvalues of A are 2 and 3. However, the stage cost penalizes only the first state component x_1 , while the second component x_2 does not appear in the state cost. Therefore, from the viewpoint of the objective function, an unstable behavior in the second state may go completely unnoticed unless it is indirectly forced to be controlled through the dynamics.

This already suggests an important principle:

Unstable modes must be visible in the cost function, otherwise the optimal controller may fail to stabilize them.

To make this statement precise, write $Q = C^\top C$. Then the cost penalizes exactly the directions that are observable through the pair (C, A) .

Stability of the optimal infinite-horizon LQR controller

Let $Q = C^\top C$. The optimal infinite-horizon feedback F_∞ stabilizes the system if and only if

1. the pair (A, B) is stabilizable;
2. the pair (C, A) has no unobservable unstable modes.

Equivalently, all unstable modes of A must be detectable through the state penalty Q .

The second condition is often stated by saying that the pair $(A, Q^{1/2})$ must be **detectable**. Detectability is weaker than observability: it does not require all modes to be observable, but only the unstable ones.

¹ P_∞ can be computed by solving the Algebraic Riccati Equation

Interpretation This theorem has a clear practical meaning. The matrix R penalizes the control effort, while Q determines which state directions the controller tries to regulate. If an unstable mode is not penalized by Q , then the optimization problem may prefer not to spend control effort on that mode, since doing so does not improve the objective. As a consequence, the optimal controller may exist but fail to stabilize the closed-loop system.

Hence, in practice:

- (A, B) stabilizable ensures that unstable modes can in principle be controlled;
- detectability of $(A, Q^{1/2})$ ensures that every unstable mode matters in the cost.

When both conditions hold, the stabilizing solution P of the algebraic Riccati equation yields the optimal feedback

$$u_t = F_\infty x_t, \quad F_\infty = -(R + B^\top P B)^{-1} B^\top P A,$$

and the closed-loop matrix $A + B F_\infty$ has all eigenvalues inside the unit disk.

2 Model Predictive Control

2.1 Introduction

In the previous chapter we have seen an overview of the LQR problem and how to solve it using dynamic programming. In that formulation, we assumed that there were no constraints on the state and control input. However, in many real-world applications, constraints on the state and input must be explicitly taken into account.

In the presence of constraints, the problem cannot, in general, be solved in closed form via backward induction as in the unconstrained LQR case, since the value function is no longer quadratic. The resulting optimization problem becomes:

Constrained LQ finite horizon problem

$$\begin{aligned} \min_{u_0, \dots, u_{T-1}, x_0, \dots, x_T} \quad & \sum_{t=0}^{T-1} (x_t^\top Q x_t + u_t^\top R u_t) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{t+1} = A x_t + B u_t, \quad t = 0, \dots, T-1 \\ & x_0 = x_0 \\ & x_t \in \mathcal{X}_t \\ & u_t \in \mathcal{U}_t \end{aligned}$$

Usually, the constraint sets are convex, e.g., polytopes.

One could solve this optimization problem in a **one-shot** fashion (i.e., open-loop strategy), obtaining an optimal input sequence $u_0, \dots, u_{T-1}$ for the given initial state x_0 , instead of a state-feedback policy as in the unconstrained case (where $u_t = \Gamma_t x_t$).

However, if disturbances or model mismatch cause the system to deviate from the predicted trajectory, the previously computed control sequence is no longer optimal. As a result, the system will follow a different trajectory than expected (see Figures 1 and 2).

To address this issue, the optimal control problem must be re-solved using the updated state information. Since we assume a Markovian system, the current state contains all the information needed to compute an optimal control sequence for the future. This implies that the optimization problem must be solved repeatedly, at each time step, within one sampling interval.

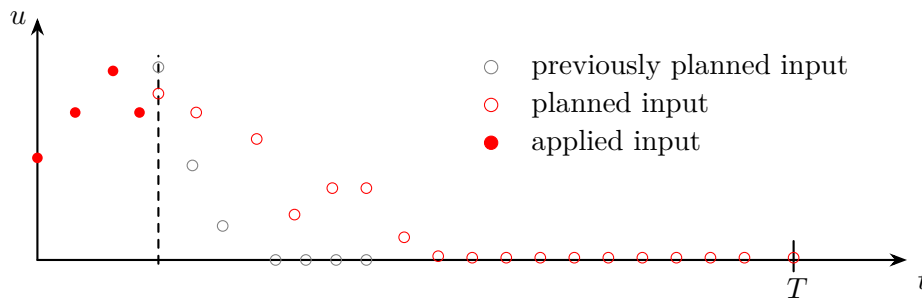


Figure 1: Receding-horizon update of the input sequence: previously planned inputs are shifted, a new optimal input sequence is computed based on the current state.

The idea that emerges from this discussion is to solve the optimization problem in a **receding horizon** fashion. At each time step, we solve a finite-horizon optimal control problem of length T , and apply only the first control input of the optimal sequence (see Figure 3).

At the next time step, the new state is measured, and the optimization problem is solved again

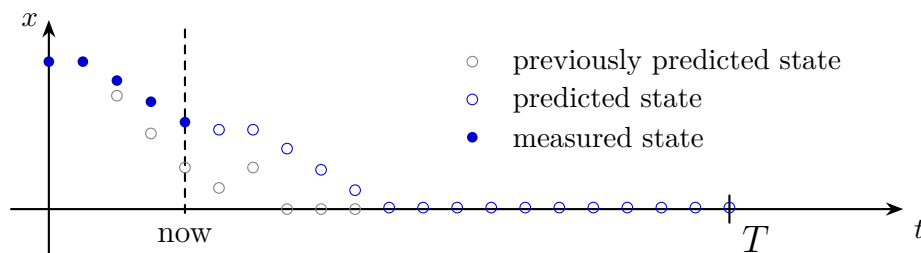


Figure 2: Receding-horizon update of the state trajectory: previously predicted states are corrected using the measured state at the current time, and a new optimal state trajectory is predicted over the horizon.

with the updated initial condition. If the system is time-invariant, the optimization problem remains the same at each time step, except for the initial state. The corresponding predicted state evolution over the horizon is illustrated in Figure 4.

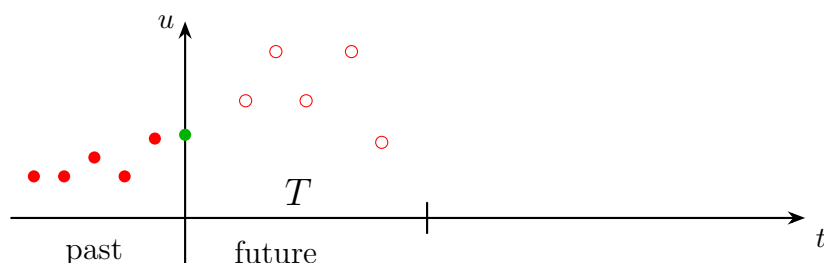


Figure 3: Finite-horizon input sequence in receding horizon control: at the current time, an optimal sequence of future inputs over a horizon of length T is computed, while past inputs are fixed.

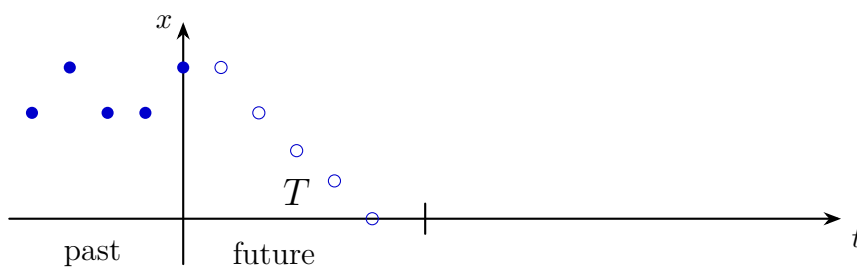


Figure 4: Finite-horizon state prediction in receding horizon control: starting from the current state, future states are predicted over a horizon of length T using the planned input sequence.

Thus the following scheme defines the receding horizon control strategy, also known as **Model Predictive Control (MPC)**:

1. At time t , measure the current state x_t .
2. Solve the finite-horizon optimal control problem with initial state x_t to obtain the optimal input sequence $u_t, u_{t+1}, \dots, u_{t+T-1}$.
3. Apply the first control input u_t to the system.
4. Increment time $t \leftarrow t + 1$ and repeat from step 1.

The resulting **open-loop optimal control problem** corresponds to a *parametric optimization problem* parametrized in the **initial state x** .

Parametric optimization problem

$u^*(x)$ is determined by^a

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{x}} \quad & \sum_{k=0}^{T-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k \\ & \mathbf{x}_0 = \mathbf{x} \\ & x_k \in \mathcal{X}_k \\ & u_k \in \mathcal{U}_k \end{aligned}$$

^aNotation: k is the index for the optimization problem, while t is the index for the time steps of the system.

The parametric nature of the problem immediately raises the question of whether the resulting control strategy is *feedforward* or *feedback*. Although the optimization problem computes an open-loop input sequence, the receding horizon implementation introduces feedback implicitly. Indeed, at each time step the optimization problem is solved using the current state x_t , and only the first input of the optimal sequence is applied. Therefore, the implemented control law can be written as

$$u_t = u^*(x_t),$$

which defines a **state-feedback control law**. One can have a visual intuition of this feedback structure by looking at the block diagram in Figure 5.

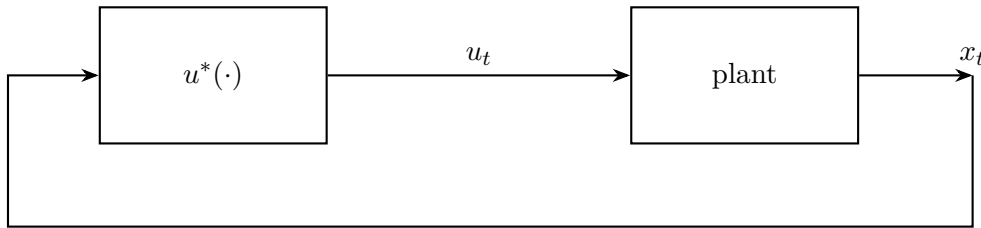


Figure 5: Block diagram of the MPC control scheme

We can establish some properties about this optimization problem:

- Under **continuity** assumptions on the cost and the dynamics, the minimum cost is a continuous function of the initial state x .
- Under **compactness** assumptions on the constraints, the parametric optimization problem admits a solution for all x in the feasible set.
- The map $x \mapsto u^*(x)$ is not necessarily continuous for a general cost function

In the case of linear dynamics, quadratic cost, and convex constraints, the optimization problem is a convex quadratic program (QP) parametrized in the initial state x . In this case, the map $x \mapsto u^*(x)$ is piecewise affine and continuous. However, in general, the control law induced by MPC can be nonlinear and non-smooth.

MPC control law

As a consequence, MPC can be interpreted as a **memory-less, nonlinear, time-invariant feedback control law**. The nonlinearity arises from the presence of constraints, even when the system dynamics are linear and the cost is quadratic.

A natural question is whether MPC can achieve *perfect tracking*, i.e., zero steady-state error. In general, this is not guaranteed without additional design choices (e.g., terminal ingredients or disturbance modeling), since the controller repeatedly solves a finite-horizon problem and does not explicitly enforce asymptotic properties beyond the prediction horizon. To better understand the relation between MPC and the finite-horizon optimal control problem, consider the following example:

Example

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{u}} \quad & \sum_{t=0}^{T-1} a^t |u_t| + |x_T|^2, \quad |a| < 1 \\ \text{s.t.} \quad & x_{t+1} = x_t + u_t. \end{aligned}$$

Two different questions can be asked:

1. What is the optimal open-loop solution $(\mathbf{x}^*, \mathbf{u}^*)$?
2. What is the MPC feedback law $x \mapsto u^*(x)$ obtained by solving the problem in a receding horizon fashion?

To solve this we observe that the cost is not quadratic and that the cost of applying inputs is discounted over time. Furthermore the state is penalized only at the end of the horizon.

Thus the optimal solution is to simply wait for the last time step and then apply $u_T = -x_T$, which yields a cost of $|x_0|^2$.

In contrast, the MPC feedback law is given by the control law $x \mapsto u^*(x) = 0$.

A key observation is that the optimal trajectory associated with the finite-horizon problem is, in general, **never realized in closed loop**, even in the absence of disturbances or model mismatch. This is because at each time step the optimization problem is re-solved with a shifted horizon, leading to a different optimal sequence than the one computed previously. ■

Consider now the unconstrained linear-quadratic problem:

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{u}} \quad & \sum_{t=0}^{T-1} (x_t^\top Q x_t + u_t^\top R u_t) + x_T^\top S x_T, \quad Q, S \succeq 0, R \succ 0 \\ \text{s.t.} \quad & x_{t+1} = A x_t + B u_t. \end{aligned}$$

Finite-time LQR yields a time-varying linear feedback law

$$\begin{aligned} u_t &= \Gamma_t x_t \\ \Gamma_t &= -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \end{aligned}$$

where the matrices P_t are computed backward in time. The control gain explicitly depends on time through P_{t+1} .

In contrast, **MPC** repeatedly solves the same finite-horizon problem at each time step. As a result, the applied control law is

$$u_t = \Gamma_0 x_t,$$

where Γ_0 is the gain associated with the first step of the horizon. Hence, MPC induces a **linear time-invariant** feedback law.

More explicitly, at time $t = 0$ both approaches yield the same control input:

$$u_0^{\text{MPC}} = -(R + B^\top P_1 B)^{-1} B^\top P_1 A x_0 = u_0^{\text{LQR}}.$$

However, at the next time step a discrepancy appears. Finite-time LQR applies

$$u_1^{\text{LQR}} = -(R + B^\top P_2 B)^{-1} B^\top P_2 A x_1,$$

while MPC recomputes the problem and applies again

$$u_1^{\text{MPC}} = -(R + B^\top P_1 B)^{-1} B^\top P_1 A x_1.$$

Therefore, even in the unconstrained case, MPC does not reproduce the finite-horizon optimal control sequence beyond the first step.

2.2 Implementation of MPC

The implementation of MPC requires solving an optimization problem at each time step. We show two main approaches to implement MPC:

1. Compute the **function** $u^*(\cdot)$ **offline**, and then evaluate it online at each time step.
 - It is extremely complex to solve with exception of very few cases
 - Abundant offline computational resources are required
 - Large memory is required to store the function $u^*(\cdot)$
2. Compute the **value** of the function $U^*(\cdot)$ **online** at each time step by solving the optimization problem.
 - It is often a tractable optimization problem.
 - It has limited time to perform computation online.

2.2.1 Explicit MPC

Explicit MPC corresponds to the first implementation strategy: instead of solving the optimization problem online at each sampling instant, we try to compute the feedback law $x \mapsto u^*(x)$ offline.

We have already seen one important case. In the **unconstrained linear MPC** case, the optimizer is simply the first element of the finite-horizon LQR solution. Therefore, the applied input can be written as

$$u_t = u^*(x_t) = \Gamma_0 x_t,$$

so the control law is linear.

The natural question is what happens when *constraints* are introduced. Consider the standard constrained linear MPC problem

Linear MPC with linear constraints

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{x}} \quad & \sum_{k=0}^{T-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \quad k = 0, \dots, T-1, \\ & x_0 = x, \\ & x_k \in \mathcal{X}, \quad \mathcal{X} = \{x \in \mathbb{R}^n : Cx \leq d\}, \\ & u_k \in \mathcal{U}, \quad \mathcal{U} = \{u \in \mathbb{R}^m : Eu \leq f\}, \end{aligned}$$

with $Q, S \succeq 0$, $R \succ 0$.

This is the most common MPC formulation: linear dynamics, quadratic cost, and polyhedral state/input constraints.

This problem is a **convex quadratic program** parametrized by the initial state x . In general, the optimizer is no longer globally linear. However, due to the quadratic cost and linear

constraints, it still has a very specific structure: the explicit MPC law turns out to be **piecewise affine**. The idea is best understood through a very simple example.

Example Consider a scalar integrator with horizon $T = 1$:

$$\begin{aligned} \min_{u_0, x_1} \quad & u_0^2 + x_1^2 \\ \text{s.t.} \quad & x_1 = x_0 + u_0, \\ & x_1 \leq 1. \end{aligned}$$

This example is useful because:

- it is a quadratic problem with a linear constraint,
- it is parametrized by the current state x_0 ,
- the equality constraint allows us to eliminate the variable x_1 ,
- the problem is convex, hence the KKT conditions are necessary and sufficient.

Using the model equation $x_1 = x_0 + u_0$, we can eliminate x_1 and rewrite the problem as

$$\min_{u_0} f(u_0) \quad \text{s.t.} \quad g(u_0) \leq 0,$$

where

$$\begin{aligned} f(u_0) &= u_0^2 + (x_0 + u_0)^2, \\ g(u_0) &= x_0 + u_0 - 1. \end{aligned}$$

The derivatives are

$$\begin{aligned} \nabla f(u_0) &= 4u_0 + 2x_0, \\ \nabla g(u_0) &= 1. \end{aligned}$$

KKT conditions for one inequality constraint

For a problem of the form

$$\min_x f(x) \quad \text{s.t.} \quad g(x) \leq 0,$$

the KKT conditions are

$$\begin{aligned} \nabla f(x) + \mu \nabla g(x) &= 0, & \mu &\geq 0, \\ g(x) &\leq 0, \\ \mu g(x) &= 0. \end{aligned}$$

The last relation is the *complementary slackness* condition.

Applying the KKT conditions to the problem gives

$$\begin{aligned} 4u_0 + 2x_0 + \mu &= 0, & \mu &\geq 0, \\ x_0 + u_0 - 1 &\leq 0, \\ \mu(x_0 + u_0 - 1) &= 0. \end{aligned}$$

The complementary slackness condition implies that two cases are possible.

Case 1: inactive constraint. Assume first that the inequality is inactive, so

$$\mu = 0.$$

Then stationarity becomes

$$4u_0 + 2x_0 = 0,$$

hence

$$u_0 = -\frac{x_0}{2}.$$

To check whether this candidate is feasible, we substitute it into the constraint:

$$x_0 + u_0 - 1 = x_0 - \frac{x_0}{2} - 1 = \frac{x_0}{2} - 1.$$

Therefore feasibility requires

$$\frac{x_0}{2} - 1 \leq 0 \quad \Longleftrightarrow \quad x_0 \leq 2.$$

So, for all initial states satisfying $x_0 \leq 2$, the optimal input is

$$u_0^* = -\frac{x_0}{2}.$$

Case 2: active constraint. Assume now that the inequality is active, so

$$x_0 + u_0 - 1 = 0.$$

Then

$$u_0 = 1 - x_0.$$

Substituting this into the stationarity condition,

$$4u_0 + 2x_0 + \mu = 0,$$

we obtain

$$\mu = -4u_0 - 2x_0 = -4(1 - x_0) - 2x_0 = 2x_0 - 4.$$

Since $\mu \geq 0$, this case is valid only if

$$2x_0 - 4 \geq 0 \quad \Longleftrightarrow \quad x_0 \geq 2.$$

Hence, for all $x_0 \geq 2$, the optimal input is

$$u_0^* = 1 - x_0.$$

Combining the two cases, the explicit solution is

Explicit solution of the toy MPC problem

$$u_0^*(x_0) = \begin{cases} -\frac{x_0}{2}, & x_0 \leq 2, \\ 1 - x_0, & x_0 \geq 2. \end{cases}$$

This example shows the main structural property of explicit MPC:

- the control law is **continuous**,
- the control law is **piecewise affine**,
- each affine expression is valid in a region where the same set of constraints is active.

The reason is the following. Once the active constraints are known, the KKT conditions reduce to a linear system in the decision variables and the multipliers. Solving that linear system gives an optimizer that depends *affinely* on the parameter x . When the active set changes, a different affine expression is obtained. Therefore, the state space is partitioned into regions, and on each region the controller is affine.

Although the explicit solution is conceptually appealing, it becomes difficult to use in large problems. In practice:

- there may be tens or hundreds of constraints,
- the number of regions may become very large,
- the offline computation required to determine all regions can be very long,
- even online, one must determine in which region the current state lies before evaluating the affine law.

As a consequence, explicit MPC is mainly attractive when the system dimension is small and the sampling time is so short that solving a quadratic program online would be too expensive. For larger systems, the most common approach is instead to solve the MPC optimization problem online at each sampling instant.

2.3 Stability

A useful way to think about MPC is as a *feedback design tool*. Once the prediction horizon T and the weights Q, R, S are chosen, the receding-horizon implementation induces a control law of the form

$$u_t = u^*(x_t).$$

Even though the optimization problem computes an open-loop input sequence, only the first input is applied and the problem is solved again at the next time step. Therefore, the implemented controller is a **static, time-invariant, nonlinear feedback law**. The nonlinearity does not necessarily come from the dynamics; it already appears because of the repeated constrained optimization.

From this perspective, the main question is no longer only how to solve the optimization problem, but whether the resulting closed-loop system is stable. There are two conceptually different ways to approach this question. The first is an *analysis* point of view: one tries to compute the control law $u^*(\cdot)$ explicitly and then study the stability of the closed-loop dynamics. In general, this is difficult, because the MPC law can be piecewise affine or more generally nonlinear and non-smooth. The second is a *design* point of view: instead of computing the feedback law explicitly, one looks for conditions on the design parameters T, Q, R, S that guarantee closed-loop stability directly. This is the more useful viewpoint in practice.

To study stability, the standard tool is Lyapunov analysis. Consider a discrete-time system

$$x_{t+1} = f(x_t),$$

and suppose that the origin $x = 0$ is an equilibrium, that is, $f(0) = 0$.

Stable equilibrium

The equilibrium $x = 0$ is said to be **stable** if

$$\forall \varepsilon > 0, \exists \delta > 0 \text{ such that } \|x_0\| < \delta \implies \|x_t\| < \varepsilon \quad \forall t \geq 0.$$

In words, if the initial condition is sufficiently close to the equilibrium, then the entire trajectory remains close to it for all future times.

Stability alone only guarantees that trajectories do not move far away from the equilibrium. A stronger notion is asymptotic stability, which additionally requires convergence to the equilibrium.

Asymptotically stable equilibrium

The equilibrium $x = 0$ is **asymptotically stable** if it is stable and

$$\lim_{t \rightarrow \infty} \|x_t\| = 0.$$

Hence, trajectories starting sufficiently close to the origin remain close to it and eventually converge to it.

The classical criterion to prove asymptotic stability is based on a Lyapunov function.

Lyapunov theorem for discrete-time systems

Assume there exists a real-valued function $W : \mathbb{R}^n \rightarrow \mathbb{R}$ such that

$$\begin{aligned} W(0) &= 0, \\ W(x) &> 0 \quad \forall x \neq 0, \\ W(f(x)) &< W(x) \quad \forall x \neq 0. \end{aligned}$$

Then the equilibrium $x = 0$ is asymptotically stable.

The idea is simple: W plays the role of an energy-like quantity. It is zero only at the equilibrium, positive everywhere else, and strictly decreases along the closed-loop trajectories. Therefore, the system is forced to move toward the origin.

This Lyapunov viewpoint explains why stability is relatively transparent for *infinite-horizon* MPC. Suppose that, at time t , the controller computes an input sequence that minimizes the infinite-horizon cost

$$V_t^\infty(x, \mathbf{u}) = \sum_{k=t}^{\infty} g_k(x_k, u_k),$$

subject to the system dynamics. If the minimum is attained, it is natural to define the optimal value function

$$W(x) = \min_{\mathbf{u}} V_t^\infty(x, \mathbf{u}).$$

Now comes the key observation. By Bellman's principle of optimality, if an input sequence is optimal from time t , then its tail is optimal from time $t + 1$ onward. In other words, after applying the first optimal input $u^*(x_t)$ and reaching the next state x_{t+1} , the remaining part of the optimal sequence is still optimal for the optimization problem starting at $t + 1$. As a consequence, along the closed-loop trajectory one obtains

$$W(x_{t+1}) = W(x_t) - g_t(x_t, u^*(x_t)).$$

Therefore, if the stage cost satisfies suitable positivity properties, for example if

$$g_t(x_t, u_t) > 0 \quad \text{for all } x_t \neq 0,$$

then

$$W(x_{t+1}) < W(x_t) \quad \text{for all } x_t \neq 0.$$

This means that the optimal value function is a natural Lyapunov candidate for the closed-loop system. Under reasonable assumptions, it follows that the origin is asymptotically stable.

This argument is very elegant, but it also hides some subtleties. First, an infinite-horizon MPC problem is infinite-dimensional, so from a computational viewpoint it is not directly implementable as stated. Second, one is implicitly assuming that the global minimum exists and is attained. Third, the stage cost must satisfy suitable properties so that the value function is positive definite and decreases strictly along the closed loop. These requirements are analogous in spirit to the assumptions encountered in infinite-horizon LQR.

The situation changes significantly for *finite-horizon* MPC. In that case, the same Lyapunov argument does not apply automatically. At time t , the controller minimizes a cost over the interval $[t, t + T]$. At time $t + 1$, however, the optimization problem is not simply the tail of the previous one: the first stage cost has disappeared, but a new term has entered at the end of the horizon. Therefore, the controller at time $t + 1$ is minimizing a *different* objective. As a result, the optimizer changes, and the finite-horizon value function is not automatically decreasing along the closed-loop trajectory.

This is the fundamental reason why closed-loop stability is immediate for infinite-horizon optimal control, but not for finite-horizon receding-horizon MPC. For finite-horizon MPC, stability must be enforced through additional design choices. This will be the starting point for the next part of the discussion.

A classical way to recover a Lyapunov argument for finite-horizon MPC is to modify the optimization problem by imposing a **terminal constraint**. The simplest choice is the *zero terminal constraint*, namely

$$x_{t+T} = 0.$$

The corresponding finite-horizon optimal control problem becomes

Finite-horizon MPC with zero terminal constraint

Given the current state x_t , define

$$\begin{aligned} W_T(x_t) &= \min_{\mathbf{x}, \mathbf{u}} \sum_{k=t}^{t+T-1} g_k(x_k, u_k) \\ \text{s.t. } & x_{k+1} = f(x_k, u_k), \quad k = t, \dots, t+T-1, \\ & x_t \text{ given,} \\ & x_{t+T} = 0. \end{aligned}$$

The intuition is very simple. By forcing the predicted state to reach the origin at the end of the horizon, we make the finite-horizon problem look more similar to the infinite-horizon one. In particular, once the terminal state is the equilibrium, the tail of the optimal trajectory can be completed in a natural way by keeping the system at the origin with zero input, provided that

$$f(0, 0) = 0.$$

This terminal condition restores the key mechanism that was missing in the plain finite-horizon case. Indeed, suppose that at time t the optimal solution of the MPC problem is

$$\{\tilde{u}_t, \tilde{u}_{t+1}, \dots, \tilde{u}_{t+T-1}\},$$

with associated state trajectory

$$\{\tilde{x}_t, \tilde{x}_{t+1}, \dots, \tilde{x}_{t+T}\},$$

where by construction

$$\tilde{x}_{t+T} = 0.$$

After applying the first control input $\tilde{u}_t$, the system reaches the state

$$x_{t+1} = \tilde{x}_{t+1}.$$

At time $t + 1$, a natural feasible candidate input sequence is then obtained by taking the *tail* of the previous optimizer and appending the zero input:

$$\{\tilde{u}_{t+1}, \tilde{u}_{t+2}, \dots, \tilde{u}_{t+T-1}, 0\}.$$

This is feasible because the previous optimal trajectory already satisfies $\tilde{x}_{t+T} = 0$, and the appended input $u = 0$ keeps the system at the equilibrium.

Therefore, the optimal cost at time $t + 1$ can be upper bounded by the cost of this feasible candidate. This gives

$$\begin{aligned} W_T(x_{t+1}) &\leq \sum_{k=t+1}^{t+T} g_k(x_k, u_k) \\ &= \sum_{k=t}^{t+T-1} g_k(x_k, u_k) - g_t(x_t, u_t) + g_{t+T}(x_{t+T}, u_{t+T}). \end{aligned}$$

Since $x_{t+T} = 0$ and we append $u_{t+T} = 0$, the last term vanishes under the standard assumption that the stage cost is zero at the equilibrium:

$$g_{t+T}(0, 0) = 0.$$

Hence,

$$W_T(x_{t+1}) \leq W_T(x_t) - g_t(x_t, u_t).$$

If, in addition, the stage cost is positive away from the origin, then for every $x_t \neq 0$ we obtain

$$W_T(x_{t+1}) < W_T(x_t).$$

This shows that the optimal value function W_T decreases along the closed-loop trajectories, and therefore it becomes a Lyapunov candidate for the closed-loop system.

Asymptotic stability with zero terminal constraint

Under the usual assumptions of feasibility, attainment of the minimum, $f(0, 0) = 0$, and positivity of the stage cost, the equilibrium $x = 0$ is asymptotically stable for the closed-loop system generated by finite-horizon MPC with terminal constraint

$$x_{t+T} = 0.$$

The proof is essentially the shifted-sequence argument above. The terminal constraint ensures that the optimal trajectory at time t can be transformed into a feasible candidate at time $t + 1$ simply by removing the first input and appending the zero input at the end. This gives a monotonic decrease of the value function, exactly as required by Lyapunov theory. We report here a more formal proof for completeness.

Proof. Let

$$W(x_t) = \min_{\mathbf{u}, \mathbf{x}} \sum_{k=t}^{t+T-1} g(x_k, u_k)$$

subject to

$$x_{k+1} = f(x_k, u_k), \quad x_t \text{ given}, \quad x_{t+T} = 0.$$

We want to compare the optimal cost at time $t + 1$ with the one at time t . By definition,

$$\begin{aligned} W(x_{t+1}) &= \min \sum_{k=t+1}^{t+T} g(x_k, u_k) \\ &= \min \left(\sum_{k=t}^{t+T-1} g(x_k, u_k) - g(x_t, u_t) + g(x_{t+T}, u_{t+T}) \right). \end{aligned}$$

By definition of the minimum, this quantity is less than or equal to the same expression evaluated at any feasible input sequence. In particular, let

$$\tilde{\mathbf{u}} = \arg \min_{\mathbf{u}, \mathbf{x}} \sum_{k=t}^{t+T-1} g(x_k, u_k)$$

be the optimizer of the MPC problem at time t , with associated state trajectory

$$\tilde{x}_{t+1}, \tilde{x}_{t+2}, \dots, \tilde{x}_{t+T}, \quad \tilde{x}_{t+T} = 0.$$

Now construct a feasible candidate input sequence for the problem at time $t + 1$ by taking the tail of the previous optimal sequence and appending the input

$$u_{t+T} = 0.$$

Since $\tilde{x}_{t+T} = 0$ and $f(0, 0) = 0$, this implies

$$x_{t+T+1} = 0,$$

so the terminal constraint is satisfied also at time $t + 1$.

Evaluating the cost at this feasible candidate yields

$$\begin{aligned} W(x_{t+1}) &\leq \sum_{k=t+1}^{t+T} g(\tilde{x}_k, \tilde{u}_k) \\ &= \sum_{k=t}^{t+T-1} g(\tilde{x}_k, \tilde{u}_k) - g(\tilde{x}_t, \tilde{u}_t) + g(\tilde{x}_{t+T}, u_{t+T}). \end{aligned}$$

Because $\tilde{x}_{t+T} = 0$ and $u_{t+T} = 0$, and assuming $g(0, 0) = 0$, we obtain

$$\begin{aligned} W(x_{t+1}) &\leq \sum_{k=t}^{t+T-1} g(\tilde{x}_k, \tilde{u}_k) - g(\tilde{x}_t, \tilde{u}_t) \\ &= W(x_t) - g(\tilde{x}_t, \tilde{u}_t) \\ &\leq W(x_t). \end{aligned}$$

Hence the optimal value function is non-increasing along the closed-loop trajectories. If moreover

$$g(x, u) > 0 \quad \forall x \neq 0,$$

then

$$W(x_{t+1}) < W(x_t) \quad \forall x_t \neq 0,$$

so W is a Lyapunov function for the closed-loop system. Therefore, the equilibrium $x = 0$ is asymptotically stable. $\square$

The zero terminal constraint is conceptually very appealing because it gives a clean and direct stability argument. However, it can also be quite restrictive. Requiring the predicted state to reach the origin exactly in T steps may be impossible for some initial conditions, especially when the horizon is short or constraints are tight. In other words, stability can be obtained, but possibly at the price of a reduced feasible set.

The zero terminal constraint is only one possible way to guarantee stability of finite-horizon MPC. More generally, several families of sufficient conditions can be used:

- choosing the prediction horizon T large enough;
- imposing a zero terminal constraint;
- imposing a suitable **terminal region** constraint for the state;
- combining terminal ingredients, such as terminal constraints and terminal penalties.

The common idea behind all these approaches is to recover, in one form or another, a Lyapunov-like decrease of the optimal cost along the closed-loop trajectories. The precise technical construction changes from one method to another, but the objective is always the same: to ensure that the receding-horizon controller does not merely optimize over the next T steps, but also induces a long-term stabilizing behavior.

When discussing MPC stability results, a few important subtleties should always be kept in mind.

First, throughout the analysis we implicitly assume that the MPC optimization problem is **feasible**. This is a nontrivial assumption. In particular, with a zero terminal constraint the problem may easily become infeasible, because not every state can be driven exactly to the origin in T steps while satisfying the constraints.

Second, future feasibility may depend on the control action selected at the current time. Thus, it is not enough to have feasibility only at the initial time: one would like feasibility to be preserved as the horizon recedes. This issue is often referred to as *recursive feasibility*, and it is closely connected to the choice of terminal ingredients.

Third, all these arguments assume that the **global minimum** of the optimization problem is found. This is automatic in the convex quadratic MPC problems discussed so far, but it becomes more delicate in more general nonlinear or nonconvex settings.

For these reasons, stability of MPC is not only a matter of optimization, but also of careful controller design. The horizon length, the stage cost, and the terminal ingredients must be selected jointly so that the resulting receding-horizon law is both feasible and stabilizing.

2.4 Disturbance rejection

So far, the MPC design has been discussed mainly in nominal conditions, that is, under the assumption that the model is exact and that no unknown perturbation acts on the system. In practice, however, disturbances are unavoidable. Depending on the application, the controller may be required to achieve different levels of robustness.

Disturbance rejection specifications

In an MPC problem, robustness against disturbances may refer to different objectives:

1. **rejection at steady state**, that is, **zero offset**;
2. **rejection during the transient**, namely a good response while the disturbance is acting and being compensated;
3. **guaranteed constraint satisfaction** despite disturbances.

These are related but distinct goals. A controller may remove the steady-state error and still react poorly during the transient. Conversely, a controller may have a reasonable transient behavior but fail to guarantee that state and input constraints are respected under uncertainty. In this subsection we focus on the first two aspects: zero steady-state offset and transient disturbance rejection.

2.4.1 Zero steady-state offset and incremental MPC

It is useful to remember that MPC is, in the end, a nonlinear static feedback law of the form

$$u_t = u^*(x_t).$$

This observation immediately suggests a possible weakness. If the plant requires a *nonzero* constant input in order to keep the state at the desired equilibrium, then a standard MPC formulation centered at the origin may fail to achieve

$$x_t \rightarrow 0.$$

In other words, a constant disturbance or a model mismatch may produce a **steady-state offset**. This issue is especially visible in the presence of:

- input disturbances,
- process noise,
- multiplicative uncertainty.

Moreover, trying to compensate this effect by choosing very small input weights may lead to poor numerical conditioning of the optimization problem. A standard remedy is to reformulate the controller in **incremental form**. Instead of using the input u_t directly as the manipulated variable, one introduces the input increment

$$\Delta u_t$$

and imposes the update law

$$u_{t+1} = u_t + \Delta u_t.$$

This is equivalent to augmenting the plant with an input integrator. If the original nominal model is

$$x_{t+1} = Ax_t + Bu_t,$$

then the augmented model becomes

$$\begin{bmatrix} x_{t+1} \\ u_{t+1} \end{bmatrix} = \begin{bmatrix} A & B \\ 0 & I \end{bmatrix} \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} 0 \\ I \end{bmatrix} \Delta u_t.$$

The optimization problem is then written in terms of Δu_t rather than u_t . This has two important consequences. First, the controller acquires an **integral action**, which is exactly what is needed to remove steady-state offsets caused by constant disturbances. Second, the input penalty in the stage cost now penalizes changes in the control signal, that is, it penalizes Δu_t rather than the absolute value of u_t . As a consequence, the controller naturally prefers smooth input profiles. Hence, the incremental formulation can be interpreted as an **integral nonlinear feedback law**: the feedback structure is still given by MPC, but the plant seen by the optimizer includes an integrator on the input channel. This is the basic mechanism used in MPC to obtain zero steady-state offset against constant disturbances.

2.4.2 Disturbance rejection during transients

Removing the steady-state error is often not enough. In many applications, the controller should also react well *while* the disturbance is affecting the system. A common strategy in MPC is based on two steps:

1. estimate the disturbance using the available measurements and the plant model;
2. correct the steady-state target using the estimated disturbance.

The idea is the following. The nominal prediction model alone cannot explain the measured plant behavior when a disturbance is present. Therefore, an additional disturbance variable is introduced and estimated online. The estimate is then used to shift the equilibrium that the MPC controller should track.

Unless better prior information is available, a very common choice is to model the disturbance as **constant**:

$$d_{t+1} = d_t, \quad d_t \in \mathbb{R}^{n_d}, \quad n_d \leq n.$$

This means that the unknown perturbation is assumed to vary slowly compared with the system dynamics, so that over the time scale of interest it can be regarded as approximately constant.

Step 1: Disturbance estimation Assume that the plant is described by the model

$$x_{t+1} = Ax_t + Bu_t + B_d d_t,$$

where d_t is an unknown disturbance entering through the matrix B_d . Since d_t is not measured, it must be estimated from the observed evolution of the state.

A simple and effective choice is a Luenberger-type disturbance observer. One first predicts the next state using the current state, the applied input, and the current disturbance estimate:

$$\hat{x}_{t+1} = Ax_t + Bu_t + B_d \hat{d}_t.$$

Then the disturbance estimate is corrected using the prediction error:

$$\hat{d}_{t+1} = \hat{d}_t + L(x_{t+1} - \hat{x}_{t+1}),$$

where L is an observer gain chosen so that

$$I - LB_d$$

is stable.

The logic is very intuitive. If the measured next state differs from the nominal prediction, the mismatch is interpreted as the effect of an unmodeled disturbance. The observer then updates $\hat{d}_t$ in order to make future predictions consistent with the measurements. In this way, the estimate gradually converges to the constant disturbance that is acting on the plant.

Step 2: correction of the steady-state target Once the disturbance has been estimated, the next step is to compute which steady state should be tracked by the controller. If the disturbance were absent, the desired equilibrium for a regulation problem would usually be the origin. However, when a constant disturbance is present, the true equilibrium generally shifts. Therefore, instead of forcing MPC to regulate the state to zero, we first compute a **disturbance-consistent steady-state target** (x_s, u_s) .

Under the constant disturbance model, the current estimate $\hat{d}_t$ is our best approximation of the steady-state disturbance. The corresponding steady-state target can be obtained by solving

Steady-state target selection

$$\begin{aligned}
& \min_{x_s, u_s} \quad \|x_s\|_{Q_s}^2 + \|u_s\|_{R_s}^2 \\
& \text{s.t.} \quad \begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = B_d \hat{d}_t, \\
& \quad x_s \in \mathcal{X}, \\
& \quad u_s \in \mathcal{U}.
\end{aligned}$$

The equality constraint is simply the steady-state balance equation for the disturbed system. Indeed, at equilibrium we must have

$$x_s = Ax_s + Bu_s + B_d \hat{d}_t,$$

which is equivalent to the constraint above. The optimization criterion is used to select, among all admissible equilibria compatible with the disturbance estimate, the one that is preferred according to the weights Q_s and R_s .

Once (x_s, u_s) has been computed, the MPC controller is no longer asked to regulate the plant to the nominal origin. Instead, it regulates the *deviations*

$$\tilde{x}_t = x_t - x_s, \quad \tilde{u}_t = u_t - u_s$$

toward zero. In this way, the controller tracks the correct equilibrium associated with the current disturbance estimate. As the estimate $\hat{d}_t$ evolves, the target (x_s, u_s) is updated accordingly. This is the basic mechanism that improves disturbance rejection during transients.

2.4.3 Advanced disturbance models

The constant-disturbance model is often a good default choice, but it is not the only possibility. A more general principle is summarized by the following statement.

Good regulator theorem

“Every good regulator of a system must be a model of that system.”

The message is that a controller can reject well only those classes of disturbances that are represented, explicitly or implicitly, in its internal model. Therefore, if better prior knowledge about the disturbance is available, that knowledge should be used.

For instance, the disturbance may be:

- constant,
- ramp-like,
- periodic.

This is precisely the viewpoint of the **internal model principle**: the closed-loop system can reject disturbances belonging to the class that has been modeled in the controller design.

If the disturbance has its own linear dynamics

$$d_{t+1} = A_d d_t,$$

then the plant can be augmented as

$$\begin{bmatrix} x_{t+1} \\ d_{t+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & A_d \end{bmatrix} \begin{bmatrix} x_t \\ d_t \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_t.$$

In this form, the disturbance is treated as an additional dynamical state. This model can then be used in two ways:

- inside the disturbance estimator;
- directly inside the MPC prediction model, instead of merely shifting the steady-state target.

This framework includes the constant-disturbance model as a special case, obtained by taking

$$A_d = I.$$

More generally, the choice of A_d determines which class of disturbances the controller is able to reject. In the spirit of the internal model principle, one integrator is associated with rejection of step-like disturbances, while more complex models are needed for ramps or periodic signals.

TO FINISH

3 Economic MPC

In the previous chapters we have always implicitly assumed that we are regulating the steady state $x_S = 0$, $u_S = 0$.

However, in many applications we are interested in regulating to a nonzero setpoint, or even tracking a time-varying reference trajectory. In particular, we typically want the plant to operate at a desired working point $x_S \neq 0$, and we can allow for a constant input u_S that compensates for constant disturbances.

If a desired equilibrium point (x_S, u_S) is known, it satisfies

$$x_S = Ax_S + Bu_S.$$

In this case, MPC can be used to stabilize the system around this point by introducing the shifted variables

$$\tilde{x}_k = x_k - x_S, \quad \tilde{u}_k = u_k - u_S.$$

Substituting into the system dynamics $x_{k+1} = Ax_k + Bu_k$, we obtain

$$\tilde{x}_{k+1} = A\tilde{x}_k + B\tilde{u}_k,$$

which has exactly the same form as the original system. Therefore, in the new coordinates, the regulation problem becomes a standard stabilization problem around the origin.

The finite-horizon MPC cost can then be written as

$$V(x_0) = \min_{\tilde{\mathbf{x}}, \tilde{\mathbf{u}}} \sum_{k=0}^{K-1} \left(\tilde{x}_k^\top Q \tilde{x}_k + \tilde{u}_k^\top R \tilde{u}_k \right) + \tilde{x}_K^\top S \tilde{x}_K.$$

For linear systems, this change of variables is **inconsequential**, since it does not modify the structure of the optimal control problem: the dynamics remain linear, the cost remains quadratic, and the constraints (if any) remain affine. As a result, regulating the system to (x_S, u_S) is equivalent to regulating the shifted system to the origin.

But how do we decide the steady state (x_S, u_S) to which we want to regulate? In some cases, this may be given by the problem specification, e.g., if we want to track a reference trajectory. In other cases, it can be derived from first principles, e.g., by solving the steady-state equations of the system.

In other cases, specifications indicate a desired steady state (x_{spec}, u_S) , but the system cannot be stabilized around it. In this case, we can use MPC to find the closest stabilizable point to the desired one, by solving the following optimization problem:

$$\begin{aligned} \min_{x_S, u_S} \quad & \|x_S - x_{\text{spec}}\|_{Q_S}^2 + \|u_S - u_{\text{spec}}\|_{R_S}^2 \\ \text{subject to} \quad & \begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_S \\ u_S \end{bmatrix} = 0 \\ & (x_S, u_S) \in \mathcal{X} \times \mathcal{U}, \end{aligned}$$

where $\mathcal{X}$ and $\mathcal{U}$ are the state and input constraint sets, respectively. The condition $\begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_S \\ u_S \end{bmatrix} = 0$ ensures that the chosen point is an equilibrium of the system, while the cost function penalizes deviations from the desired specifications.

This optimization problem is typically solved offline and serves to compute a feasible and stabilizable steady-state target (x_S, u_S) . The resulting pair is then used as the reference for the MPC regulator, which is responsible for driving the system to this point while satisfying

constraints.

Many modern control problems present **complex performance metrics** that cannot be easily captured by a quadratic cost function. For example, in chemical processes, the economic performance of the system may depend on nonlinear functions of the state and input variables, such as production rates, energy consumption, or profit margins. In such cases, we can use **economic MPC**, which allows for more general cost functions that directly reflect the economic objectives of the system.

Thus, let $\ell(x, u)$ be a general stage cost function that captures the economic performance of the system like economic losses, energy use, cost of material, etc. The economic MPC problem assumes that

- $\ell(x, u)$ is a continuous function;
- $\ell(x, u)$ is lower bounded in the domain

While remember that in MPC we were assuming that

- $g(x_S, u_S) = 0$ (i.e., the system is at equilibrium at the desired setpoint);
- $g(x, u) \geq 0$
- $g(x, u)$ is radially unbounded, continuous, zero at the origin, and strictly increasing.
- e.g. $g(x, u) = x^\top Qx + u^\top Ru$.

One possible approach is to separate the economic objective from the dynamic control problem. In particular, the economic cost is handled at the steady-state level, while MPC is used to steer the system toward the corresponding optimal operating point.

Given a general stage cost $\ell(x, u)$, we first define the steady-state optimization problem

$$\begin{aligned} \min_{x_S, u_S} \quad & \ell(x_S, u_S) \\ \text{subject to} \quad & x_S = f(x_S, u_S), \\ & (x_S, u_S) \in \mathcal{X} \times \mathcal{U}. \end{aligned}$$

This problem determines the economically optimal equilibrium of the system.

The resulting pair (x_S, u_S) is then used as a reference for the MPC controller, which is designed to track this steady state using a standard quadratic cost. In this way, the economic objective is indirectly enforced through steady-state optimization, while the MPC ensures closed-loop stability and constraint satisfaction.

This approach is typically implemented by solving the steady-state optimization problem offline, or at a slower time scale than the MPC loop. However, this separation introduces some limitations. The closed-loop trajectory is generally suboptimal from an economic point of view, since the stage cost $\ell(x, u)$ is not minimized during transients. Moreover, the controller cannot exploit transient behavior to improve performance, and its response to disturbances may be economically inefficient. On the other hand, the resulting MPC problem remains a standard linear-quadratic program, which is computationally efficient and well understood.

An alternative approach is to incorporate the economic objective directly into the dynamic optimization problem. This leads to the formulation of **economic MPC**, in which the stage cost $\ell(x, u)$ is optimized along the entire predicted trajectory:

$$V(x) = \min_{\mathbf{x}, \mathbf{u}} \sum_{k=0}^{K-1} \ell(x_k, u_k)$$

subject to

$$x_{k+1} = f(x_k, u_k), \quad x_0 = x, \quad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}.$$

In contrast to the previous approach, there is no separation between steady-state optimization and tracking. The controller directly optimizes the economic performance over the prediction horizon, resulting in a *single time-scale* scheme in which both transient and steady-state behavior are determined simultaneously.

As a consequence, the controller computes control actions that generate an economically optimal trajectory, rather than simply driving the system toward a precomputed steady state. In particular, the optimal operation may not correspond to a fixed equilibrium, and the controller may intentionally exploit transient behavior if this improves the overall performance. This increased flexibility comes at the price of a more challenging optimization problem. Since the cost $\ell(x, u)$ is in general nonlinear and not necessarily convex, the resulting optimal control problem may be non-convex and harder to solve in real time. Moreover, standard quadratic Lyapunov arguments used in tracking MPC no longer apply directly, and additional conditions are required to guarantee stability.

On the other hand, economic MPC is particularly well suited for nonlinear systems and applications in which performance is naturally described by non-quadratic criteria, such as energy efficiency or economic profit. In these cases, directly optimizing $\ell(x, u)$ allows the controller to fully exploit the system dynamics and constraints to achieve improved closed-loop performance.

A key issue with economic MPC arises from the fact that the stage cost $\ell(x, u)$ is not designed to enforce convergence to a steady state. In particular, it may happen that there exist pairs (x, u) that are not equilibria of the system, i.e.,

$$x \neq f(x, u),$$

for which

$$\ell(x, u) < \ell(x_S, u_S),$$

where (x_S, u_S) is an optimal steady state.

This situation is not pathological; on the contrary, it is quite common in practice. Since $\ell(x, u)$ reflects instantaneous economic performance, it may be advantageous for the system to operate away from equilibrium, for instance by exploiting transient dynamics or time-varying operating conditions.

As a consequence, convergence to a steady state is no longer guaranteed nor even desired in general. The optimal closed-loop behavior may correspond to trajectories that do not settle at an equilibrium, such as periodic or more complex operating regimes. This highlights a fundamental difference with standard tracking MPC: convergence to a steady state is not part of the control specification. Instead, the objective is to optimize economic performance over time, possibly at the expense of steady-state operation.

However, this increased flexibility comes with important challenges. First, operating away from equilibrium can lead to behaviors that are harder to analyze and predict. In particular, stability and boundedness of the closed-loop trajectories are not automatically guaranteed. Second, without additional conditions or constraints, the controller may generate trajectories that are economically optimal in the short term but undesirable from a safety or robustness perspective.

A fundamental design choice in economic MPC is the length of the prediction horizon. In principle, the most natural formulation is the infinite-horizon problem

$$\sum_{k=0}^{\infty} \ell(x_k, u_k),$$

which directly captures the long-term economic performance of the system.

The infinite-horizon formulation has a clear economic interpretation, as it reflects the cumulative cost over an indefinite future. This is particularly meaningful in continuous operation settings, where there is no natural terminal time and performance is evaluated over the long run. Moreover, constraints are enforced over the entire trajectory, which ensures boundedness of the closed-loop system by construction.

However, the infinite-horizon problem is infinite-dimensional and therefore computationally intractable in practice. For this reason, it cannot be solved directly in real time.

As a result, practical implementations rely on a finite-horizon approximation of the form

$$\sum_{k=0}^{K-1} \ell(x_k, u_k).$$

This leads to a tractable optimization problem that can be solved online within the MPC framework.

In standard tracking MPC, a finite horizon provides a good approximation of the infinite-horizon problem, since the system is driven toward an equilibrium and the contribution of the tail beyond the horizon can be neglected or approximated. In economic MPC, this argument no longer holds. Since the optimal behavior may not converge to a steady state, the portion of the trajectory beyond the horizon can have a significant impact on performance.

This mismatch introduces several potential issues. First, the controller may favor trajectories that yield low cost within the prediction horizon but lead to poor behavior afterward, as future consequences are not accounted for. In particular, the system may move toward regions from which recovery is difficult or expensive, resulting in undesirable or even unstable behavior. Second, feasibility problems may arise at the end of the horizon. Due to state and input constraints, not all states reached within the horizon can be extended to admissible infinite trajectories. This can lead to situations in which the optimization problem becomes infeasible at future time steps.

A common approach to mitigate these issues is to augment the finite-horizon problem with a terminal constraint. In particular, one enforces that the predicted trajectory reaches a feasible steady state at the end of the horizon, i.e.,

$$x_K = x_S, \quad u_K = u_S,$$

where (x_S, u_S) is an admissible equilibrium of the system.

This modification preserves the computational tractability of the finite-horizon formulation, while introducing a mechanism to guarantee that the predicted trajectory can be extended beyond the horizon in a feasible way. In other words, by constraining the terminal state to an equilibrium, one ensures that there exists a feasible infinite-horizon continuation, thereby recovering *recursive feasibility*.

From this perspective, the terminal constraint provides an explicit characterization of what constitutes a feasible infinite-horizon trajectory: a trajectory is admissible if it can be steered to a steady state that satisfies the system dynamics and constraints.

However, the introduction of a terminal constraint also raises important questions regarding the resulting closed-loop behavior. In particular, it is no longer immediate whether the closed-loop system is stable, or whether the resulting trajectory achieves optimal economic performance. The terminal constraint enforces convergence to a steady state, which may restrict the ability of the controller to exploit economically favorable transient or non-equilibrium operation.

Finally, it is important to emphasize that, as in any MPC scheme, the closed-loop system does not follow the predicted trajectory exactly. At each time step, only the first control input is applied,

$$u_k = u^*(x_k),$$

and the optimization problem is solved again at the next step with updated state information. As a result, the actual closed-loop trajectory is generated by the repeated solution of the finite-horizon problem, and not by the open-loop prediction computed at a single time instant.

To better illustrate the difference between standard tracking MPC and economic MPC, consider the following linear system

$$x_{k+1} = Ax_k + Bu_k, \quad A = \begin{bmatrix} \frac{1}{2} & 1 \\ 0 & \frac{3}{4} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

with input constraint

$$u \in \mathcal{U} = [-1, 1],$$

and economic stage cost

$$\ell(x, u) = q^\top x + ru, \quad q = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, \quad r = -10.$$

The first step consists in computing the optimal steady state by solving

$$\begin{aligned} \min_{x_S, u_S} \quad & \ell(x_S, u_S) \\ \text{subject to} \quad & x_S = Ax_S + Bu_S, \\ & u_S \in \mathcal{U}. \end{aligned}$$

To find the steady states we need to solve

$$(I - A)x_S = Bu_S,$$

which gives

$$\begin{bmatrix} \frac{1}{2} & -1 \\ 0 & \frac{1}{4} \end{bmatrix} \begin{bmatrix} x_{S,1} \\ x_{S,2} \end{bmatrix} = \begin{bmatrix} 0 \\ u_S \end{bmatrix}$$

Since the cost is linear and the constraints define a polyhedral set, this is a linear program, and the optimum is attained at an extreme point of the feasible set. Thus we only need to evaluate the cost at the two vertices of the input constraint, which gives

$$\text{if } u_S = -1, \quad x_S = \begin{bmatrix} -4 \\ -8 \end{bmatrix}, \quad \ell(x_S, u_S) = 18,$$

$$\text{if } u_S = 1, \quad x_S = \begin{bmatrix} 4 \\ 8 \end{bmatrix}, \quad \ell(x_S, u_S) = -18.$$

The solution (x_S^*, u_S^*) represents the economically optimal equilibrium. Once (x_S^*, u_S^*) is determined, two fundamentally different control strategies can be constructed.

In the standard MPC tracking approach, the optimal steady state is used as a reference. Defining the shifted variables

$$\tilde{x}_k = x_k - x_S^*, \quad \tilde{u}_k = u_k - u_S^*,$$

the controller minimizes a quadratic tracking cost of the form

$$g(\tilde{x}, \tilde{u}) = \tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u},$$

driving the system to (x_S^*, u_S^*) as fast as possible. A terminal constraint $\tilde{x}_K = 0$ (equivalently $x_K = x_S^*$) is typically imposed to ensure stability. The resulting optimization problem is a quadratic program.

In contrast, economic MPC directly uses the economic cost in the receding horizon problem:

$$\min_{\mathbf{x}, \mathbf{u}} \sum_{k=0}^{K-1} \ell(x_k, u_k)$$

subject to

$$x_{k+1} = Ax_k + Bu_k, \quad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U},$$

together with a terminal constraint $x_K = x_S^*$ to guarantee feasibility. In this case, since both the dynamics and the cost are linear, the resulting problem is a linear program.

The key difference between the two approaches lies in the objective being optimized. Tracking MPC enforces convergence to the economically optimal steady state and prioritizes stability and regulation performance, but does not optimize the economic cost during transients. Economic MPC, on the other hand, optimizes the economic objective along the entire trajectory, potentially exploiting transient behavior to improve performance.

From a computational perspective, tracking MPC leads to a quadratic program, which is typically well-conditioned and easier to solve reliably in real time. Economic MPC may lead to linear or nonlinear programs depending on the form of $\ell(x, u)$, and can be more challenging, although in this particular example it reduces to a linear program.

Now we want to analyze the closed loop behavior. In particular we want to understand if the closed loop stability is guaranteed and if the economic cost is minimized. In standard tracking MPC, the terminal constraint plays a crucial role in proving asymptotic stability. Indeed, when the cost is built from a positive definite tracking stage cost

$$g(\tilde{x}, \tilde{u}),$$

with

$$g(0, 0) = 0, \quad g(\tilde{x}, \tilde{u}) > 0 \quad \text{for } \tilde{x} \neq 0,$$

the finite-horizon optimal value function can be used as a Lyapunov function for the closed-loop system. The key argument is the same shifted-sequence argument already discussed in the previous chapter. If the optimal predicted trajectory satisfies the terminal constraint

$$\tilde{x}_K = 0,$$

then, after applying the first optimal input, a feasible candidate sequence at the next time step is obtained by shifting the previously optimal sequence and appending the equilibrium input $\tilde{u} = 0$. Since the appended terminal stage cost is zero, one obtains

$$W(f(\tilde{x}, u_0^*(\tilde{x}))) \leq W(\tilde{x}) - g(\tilde{x}, u_0^*(\tilde{x})) \leq W(\tilde{x}),$$

which shows that W decreases along the closed-loop trajectories and is therefore a valid Lyapunov function.

The natural question is whether the same idea can be applied to economic MPC. Suppose that we consider the finite-horizon economic MPC problem with terminal constraint

$$x_K = x_S,$$

where (x_S, u_S) is an admissible steady state. The same shifted-sequence construction remains feasible: after applying the first optimal control input, one can shift the optimal predicted trajectory and append the steady-state input u_S at the end of the horizon. Therefore, feasibility is preserved. However, the cost argument changes in a fundamental way.

Indeed, for economic MPC the stage cost is $\ell(x, u)$, which is not positive definite with respect to the steady state and is not, in general, minimized at (x_S, u_S) . As a consequence, the shifted-sequence argument only yields

$$W(f(x, u_0^*(x))) \leq W(x) - \ell(x, u_0^*(x)) + \ell(x_S, u_S).$$

This inequality is not sufficient to conclude that W decreases. In fact, it may happen that

$$\ell(x_S, u_S) > \ell(x, u_0^*(x)),$$

so the right-hand side can be larger than $W(x)$. Therefore, unlike in tracking MPC, the optimal finite-horizon value function is *not* automatically a Lyapunov function for the closed-loop system.

This observation has an important conceptual consequence. In economic MPC, asymptotic stability of the steady state is not guaranteed by the terminal constraint alone. The terminal constraint guarantees recursive feasibility, but not necessarily convergence to the equilibrium. This is perfectly consistent with the economic interpretation of the problem: the controller is asked to optimize economic performance, not to minimize the distance from the steady state. If operating away from equilibrium is economically more favorable, then the closed-loop system may prefer persistent transient behavior or even non-equilibrium operating regimes.

For this reason, asymptotic convergence to the steady state is not always the most relevant objective in economic MPC. In some applications, one may actually achieve a lower long-run cost by not converging to an equilibrium. For nonlinear systems, it is even possible that economically preferable periodic trajectories exist, so that oscillatory operation can outperform steady-state operation. Hence, from the viewpoint of economic MPC, the relevant performance question is often not whether the state converges to x_S , but whether the closed-loop system achieves good *average* economic performance over time.

This leads to a weaker, but very meaningful, notion of closed-loop performance. Consider the closed-loop trajectory generated by economic MPC with terminal constraint x_S . Under suitable assumptions, one can prove the following asymptotic average performance bound:

$$\limsup_{t \rightarrow \infty} \frac{1}{t} \sum_{k=0}^{t-1} \ell(x_k, u_k) \leq \ell(x_S, u_S).$$

This result states that the asymptotic average economic performance of the closed loop is no worse than the cost of operating permanently at the chosen steady state. In other words, even if the trajectory does not converge to (x_S, u_S) , its long-run average performance is at least as good as the steady-state economic benchmark.

The proof again relies on the terminal constraint and on the shifted-sequence argument. From feasibility of the shifted trajectory, one obtains

$$W(f(x, u_0^*(x))) \leq W(x) - \ell(x, u_0^*(x)) + \ell(x_S, u_S).$$

Summing this inequality along the closed-loop trajectory for $k = 0, \dots, t-1$ yields

$$\sum_{k=0}^{t-1} \ell(x_k, u_k) \leq t \ell(x_S, u_S) + W(x_0) - W(x_t).$$

Dividing by t gives

$$\frac{1}{t} \sum_{k=0}^{t-1} \ell(x_k, u_k) \leq \ell(x_S, u_S) + \frac{W(x_0) - W(x_t)}{t}.$$

If the value function remains bounded along the closed loop, the last term vanishes as $t \rightarrow \infty$, from which the average performance bound follows.

It is important to stress the meaning of this result. The function W is not a Lyapunov function here, because it is not required to decrease at every time step. Instead, it acts as a storage-like quantity that allows us to compare the accumulated closed-loop cost with the steady-state benchmark. Thus, the conclusion is not asymptotic stability, but an *asymptotic average cost guarantee*.

We can now summarize the main message of this discussion. In economic MPC, a terminal constraint is essential to guarantee recursive feasibility of the receding-horizon scheme. However, unlike in tracking MPC, the terminal constraint alone is not enough to guarantee asymptotic stability of the optimal steady state. The closed-loop system may generate economically efficient trajectories that do not converge to an equilibrium, and this is not necessarily undesirable. In fact, even when the trajectory eventually converges to the steady state, economic MPC may still outperform standard tracking MPC because it optimizes the transient behavior as well. Over long operating periods, and especially in the presence of recurrent disturbances, this transient economic advantage can accumulate and lead to significantly improved overall performance.

Additional assumptions can be introduced to recover asymptotic stability of the steady state in economic MPC. These conditions are stronger than those used in standard tracking MPC and are not automatic consequences of the basic formulation. Their role is precisely to connect economic optimality with a suitable notion of dissipativity or storage of the system, so that the closed-loop behavior can again be related to equilibrium convergence. In the absence of such additional assumptions, the most natural guarantee remains the asymptotic average performance property discussed above.